



Decision Theory II

Reflective Oracles, Open-Source Games, and Commitment Races

Daniel C



Where we are: agents that model each other

Two foundational obstacles to mutual modelling

- Deck I ended at **fair computational worlds**: agent and environment are both programs that may inspect, simulate, or predict each other. Embedded agents must model other agents – including agents like themselves.
- This raises two questions, and this deck takes both:
 - (1) **Can a Bayesian agent even represent other agents like itself?** Self-modelling failures of AIXI, the *grain of truth* problem, and *reflective oracles*.
 - (2) **When agents can read each other's code, how do they cooperate without conflict?** Open-source game theory, *commitment races*, and *safe Pareto improvements*.
- Punchline preview: reflective oracles solve mutual modelling *semantically* (a shared oracle); open-source games solve it *syntactically* (read the code). Both then face the strategic problem of *commitment*.



Large agents vs. embedded agents

The cybernetic picture and its hidden assumptions

- Cybernetic model: an agent learns a *model* of its environment and *plans* with it towards a goal. Two implicit assumptions:
 - (1) the agent is **“larger than”** the environment – its mind can hold a full copy of the world;
 - (2) the agent is **“outside”** the environment. (Note $1 \Rightarrow 2$, so model-based RL almost always assumes (2).)
- Embedded reality: the agent is a *special part* of the environment. It can be modified, copied, or destroyed; and **other equally powerful agents may exist, mutually reasoning about one another.**
- Can we still reason when the agent is *smaller* than its environment, and contained in it?

(This section follows C. Wyeth, *Embedded Agency* / Wyeth & Hutter.)



AIXI: the ideal large agent

Optimal planning against a universal mixture

- History-based RL: model very general tasks as an agent emitting actions and receiving percepts/rewards over time.
- **AIXI** (Hutter) is the optimal history-based agent for *computable* environments. It plans optimally against a **Bayesian mixture over all programs**, weighting each candidate environment by its simplicity (Solomonoff prior, weight $\sim 2^{-\text{length}}$).
- One-line idea: *try every program that could be the world, weight it by simplicity, and act to maximise expected reward under the mixture.*
- AIXI is the gold standard of an idealised *dualistic* agent. The question of this section: is it ready to be embedded?



AIXI cannot model itself

“AIXI does not believe the universe contains AIXI”

- To go beyond the cybernetic split, try **jointly predicting the whole history** (actions *and* percepts) with a Solomonoff mixture. Problem: joint prediction does not transfer cleanly to RL, and learning may fail to converge even when the environment is computable.
- AIXI believes its environment is lower semi-computable, but **AIXI's own optimal policy is not lower semi-computable** (it sits higher in the computational hierarchy). So AIXI's hypothesis class *cannot contain a copy of itself*.
- **Self-modelling failure** (Wyeth–Hutter): there are histories where, even though the reward has *always* matched the value of the action bit, AIXI *infinitely often doubts* the pattern will continue (failure of “prediction of selected bits”). Normalising the mixture repairs some, but not all, of this.



The grain of truth problem

Bayesian agents that are never “infinitely surprised”

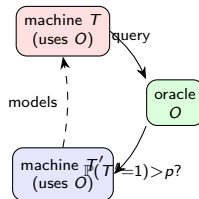
- In a multi-agent game, agents modelling each other must be of **the same “size”**. So AIXI cannot learn an environment that contains other AIXIs – mutual modelling breaks the large-agent assumption.
- **Grain of truth property:** each player’s prior should assign *positive probability to the truth* (the other agents’ actual policies) – it should never be *infinitely surprised* by an event that truly has positive probability. (Formally: the *true* distribution is absolutely continuous w.r.t. the agent’s beliefs.)
- Naive constructions fail: if policies must *run each other* to predict each other, the mutual calls **do not halt** (infinite regress). We need a way for same-size agents to predict same-size agents *without hanging*.



Reflective oracles

Self-reference without paradox, via randomisation

- Give each machine a **reflective oracle** O : it answers queries “is $\mathbb{P}(\text{machine } T \text{ outputs } 1) > p$?” for machines T that may themselves call O .
- The liar machine “output 0 iff O says I output 1 with prob. $> \frac{1}{2}$ ” would contradict a deterministic oracle. A reflective oracle is allowed to **randomise** on such borderline cases (when the true probability equals the threshold), so no machine hangs and no contradiction arises.
- Reflective-oracle-computable strategies are *closed under mutual modelling*, so a **reflective AIXI** satisfies the grain of truth property. Consequences: convergence to Nash equilibria in repeated games; with joint prediction, **acausal coordination** in the Prisoner’s Dilemma; the self-modelling/semimeasure failures vanish.
- **Bridge**: reflective oracles solve mutual modelling *semantically* (a shared oracle). Open-source game theory does it *syntactically* – the agents simply read each other’s source code.



randomise on the diagonal $\Rightarrow$ no liar paradox



A 60-second refresher on game theory

Nash, and the Prisoner's Dilemma

- A (normal-form) game: players, action sets, and payoffs $u_i(a_1, \dots, a_n)$ depending on the *joint* action.
- **Nash equilibrium:** a profile where no player gains by deviating *alone*.
- **Prisoner's Dilemma:** (C, C) is better for both than (D, D) , yet D *dominates* C for each player. The unique Nash is (D, D) : rational players defect, and both end up worse.
- **Root cause:** your payoff depends on the *joint* action, but you must choose *without knowing or affecting* the other's choice. You cannot *condition* on what they do.

		Player 2	
		C	D
P1	C	3, 3	0, 5
	D	5, 0	1, 1

(D, D) is the only Nash, though (C, C)
beats it for both.



Löbian cooperation: FairBot

When players can read each other's code

- **Open-source (program) game** (Tennenholtz): each player submits a *program* that takes the opponent's *source code* as input and outputs an action. Now you *can* condition on the opponent.
- **FairBot**: “cooperate iff I can *prove* my opponent cooperates with me.”
- **FairBot vs. FairBot**: by **Löb's theorem**, each proves mutual cooperation and so cooperates. (Here Löb is a *feature*: $\Box(\Box C \rightarrow C) \rightarrow \Box C$, the self-referential proof search succeeds.) Two mutually suspicious agents cooperate, just by reading each other.
- **Unexploitable**: FairBot cooperates with CooperateBot, but *defects* against DefectBot (it cannot prove a defector cooperates). It is never the sucker.
- Caveat: proof-based FairBot is *brittle* – it depends on syntactic provability, can fail to prove cooperation with a differently-written but behaviourally-cooperative bot, and proof search may not terminate.



Robust cooperation by simulation: ε -grounded bots

Replacing proofs with simulation (Oesterheld)

- **ε -grounded FairBot:** with small probability ε , cooperate *unconditionally* (“ground out”); with probability $1 - \varepsilon$, *run the opponent’s program* on your own code and *copy* its action.
- **Why it terminates:** each level of mutual simulation grounds out with probability ε , so the recursion halts with probability 1 – no infinite regress (contrast the naive “run each other” that hangs).
- Two ε -grounded FairBots (almost always) cooperate, and this is a **Nash equilibrium** of the program game. It is **robust**: programs that *behave* the same are treated the same (unlike syntactic proof-based bots).
- (The ε -grounded π Bot and its generalisations, arXiv:2412.14570, prove folk theorems for such simulation-based equilibria.)



What open-source game theory is really doing

Conditional commitment

- Two desiderata for a good decision theory here:
 - (1) **cooperate with your own copy**, and
 - (2) **unexploitability** – an opponent cannot become better off by making you worse off.
- Why ordinary game theory fails: payoff depends on the joint action, but you are uncertain about the opponent's action and they about yours – a chicken-and-egg of *mutual uncertainty*.
- **Open-source game = conditional commitment**: “I commit to X conditional on you committing to Y.”
 - the **conditional** part keeps you *unexploitable* (you cooperate only if they do);
 - the **commitment** part makes your cooperation *legible*, so you can satisfy the conditions of the opponent's conditional commitment.
- FairBot is exactly this: “I commit to C conditional on a proof that you commit to C.”



The commitment races problem

Why consequentialists race to commit (Kokotajlo)

- A consequentialist (acts purely on expected consequences) is both a **bully and a coward**: it threatens when threats work, and *caves to credible threats*. So **whoever commits first controls the one who commits later**.
- This creates a **race**: it pays to commit *before* you have thought or learned – to “move first”. For agents that read or simulate each other, even *thinking longer* leaks exploitable information about what you will do.
- Two catastrophe modes:
 - **Incompatible commitments lock in**: both players throw out the steering wheel in a game of Chicken $\Rightarrow$ a crash neither wanted (e.g. grim-trigger protocols keyed to incompatible bargaining solutions).
 - **Wasteful racing**: everyone commits before learning – a coordination failure like an arms race.



Moving first in logical time; and s-risks

Why this is a safety priority

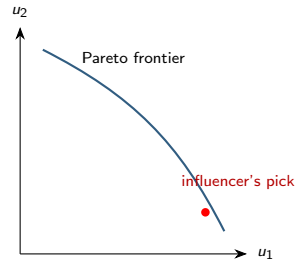
- “First” is not clock time but **logical time**: who, in the structure of mutual prediction, effectively decides *before* the other. An agent that is *predicted* to have committed *is* committed – even simultaneously in clock time.
- **Connection to s-risks (risks of astronomical suffering)**. The most dangerous commitments are *threats*: “do X or I make things very bad for you.” Commitment races can lock superintelligences into *carrying out* catastrophic threats (or executing them to build credibility), i.e. suffering on an astronomical scale.
- So a decision theory that avoids commitment races – letting agents reach good outcomes *without* a destructive scramble to pre-commit – is a genuine safety goal, not a curiosity.



Influencer vs. responder

The first-mover's advantage, and the race it triggers

- **Responder:** run the opponent's program, observe its action, then $\arg \max$ your own utility against that action (a plain *best response*).
- **Influencer:** *assume the opponent is a responder* (will best-respond to you); $\arg \max$ your utility over that – i.e. pick the point on the Pareto frontier **best for you and worst for the opponent**, knowing they will cave.
- **"The best response is not the best response":** being a naive best-responder is *dominated* by being the influencer. So everyone wants to be the influencer $\Rightarrow$ a race to "move first in logical time".



moves first $\Rightarrow$ grabs its favourite corner of the frontier



The generalized commitment race: entanglement

Why agents are not incentivised to make Pareto improvements

- When you choose your program in an open-source game, you optimise against your *subjective distribution* over the opponent's possible programs – including **counterfactual** ones (programs they might have submitted but didn't).
- So you cannot simply maximise against the opponent's *actual* program: doing so might make you worse off against *counterfactual* versions. Your incentive is **entangled** with those counterfactual programs.
- Equivalently: whenever a rational agent “*punishes*” an opponent (for, say, proposing a Pareto improvement), it is *forgoing utility against the actual program* to gain utility against some *counterfactual* program.
- **Entanglement is the obstacle:** it is exactly what disincentivises accepting a Pareto improvement (accepting might look “soft” and cost you against a counterfactual hawk). This is the *generalized* commitment race – it persists even with full conditional commitment.



Safe Pareto improvements (SPI)

Improving on a default that risks conflict

- Setup (Oosterheld–Conitzer; DiGiovanni–Clifton–Macé, arXiv:2403.05103): players delegate to *representative* programs. The *default* instruction is “play the original game as best you can”.
- An **SPI** is a re-instruction (restrict the strategy set, or re-map utilities) that is **guaranteed to make every player weakly better off** – with *no* probabilistic assumptions about how the representatives play. A Pareto improvement that is *safe* because it improves *regardless* of the opponent.
- Example: a game heading for costly conflict (Chicken-style). Agree on a transformation that strips out the mutually destructive outcome; each player does at least as well, whatever happens.
- The paper proves a clean guarantee (each player gets at least their lowest payoff in any *efficient* outcome) – but under a strong assumption.



The assumption SPI relies on – and the cheating problem

Participation independence

- **Participation independence:** representatives behave the *same* in the SPI-transformed game as in the default game – their play does not change just because an SPI is in effect. This independence is what lets you *guarantee* the improvement.
- But it is unrealistic. **The cheating problem:** if I know that an SPI will Pareto-improve away any conflict, then conflict is *less costly* for me, so I am incentivised to submit a *more hawkish* default program.
- Anticipating my hawkishness, my opponent's incentive to offer the SPI in the first place is undermined. **The SPI can defeat itself** – precisely the entanglement effect, reappearing.



Entanglement-free SPI: the idea

Make the improvement with an entanglement-free program

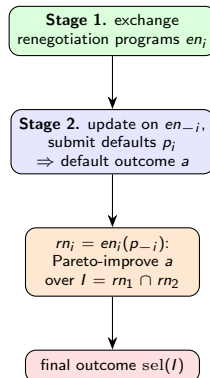
- Diagnosis: *entanglement* (caring about counterfactual opponent programs) is what disincentivises Pareto improvements. So make the improvement using a separate program that is **entanglement-free** – the **renegotiation program**.
- **Sequential structure:** agents *first* exchange renegotiation programs and *update on the actual one*; *then* submit their default programs.
- Because you update on the opponent's *actual* renegotiation program, you optimise against *it*, not against counterfactual versions $\Rightarrow$ *no entanglement* $\Rightarrow$ no incentive to punish/reject improvements.
- The renegotiation program may *only propose Pareto improvements* (conditional on the opponent's default program). Two consequences:
 - it can *never make the opponent worse off* $\Rightarrow$ no influencer-responder exploitation, *even though* it moves first;
 - it *conditions on the default program* $\Rightarrow$ it can *detect* hawkish “cheating” defaults and withhold improvements, restoring unexploitability and **removing reliance on participation independence**.
- You keep your full action space: you can still play the open-source game with your default program; the renegotiation program only *adds* the option to Pareto-improve.



Entanglement-free SPI: the protocol

A clean formalism

- Open-source game: actions $A = A_1 \times A_2$, utilities $u_i : A \rightarrow \mathbb{R}$, programs $p_i : P_{-i} \rightarrow A_i$, joint action $a = (p_1(p_2), p_2(p_1))$.
- **Renegotiation function** $rn_i : RN_{-i} \times A \rightarrow \mathcal{C}(A)$: conditional on the opponent's renegotiation function, it proposes a set of Pareto-improvements on the default outcome a . A rule sel picks one point of the agreed intersection.
- **Entanglement-free renegotiation program** $en_i : P_{-i} \rightarrow RN_i$ reads the opponent's default program, outputs a renegotiation function.
- The intersection $I = rn_1(rn_2, a) \cap rn_2(rn_1, a)$ holds the Pareto-improvements *both* agents will make; $\text{sel}(I)$ is the final outcome. (Choosing sel adds no coordination problem, under mild conditions – DiGiovanni.)





Why first-mover is safe here – and what we trade away

Resolving the race without creating a new one

- Usually, a sequential structure creates an influencer-responder race (the first mover grabs the frontier). Here the first move (renegotiation) can *only Pareto-improve*, so it **cannot exploit** – no new commitment race is introduced.
- Net result: entanglement-free SPI **solves the cheating problem** and **drops the participation-independence assumption** (the renegotiation program conditions on the defaults to catch cheating), while the sequential structure **avoids introducing its own commitment race**.
- **The trade-off:** the original SPI's clean minimum-payoff bound relied on participation independence. Dropping that assumption, entanglement-free SPI *cannot prove the same bound*. Weaker (more realistic) assumptions buy a weaker guarantee.

(Entanglement-free SPI: Daniel C; selection-function coordination after DiGiovanni.)